ET AI HACKATHON 2026

COMPLETE WINNING BLUEPRINT

Team Strategy | Architecture | Implementation Guide | Submission Plan

Selected Topic: #9 — AI Money Mentor

Sub-Theme: FinSaarthi — India's First Agentic Financial Life OS

Hackathon Partner: Unstop | Industry Partner: Avataar.ai

Generated: March 2026 | Confidential — Team Use Only

1. Topic Selection — The Winning Choice

1.1 Why Topic 9 Beats Every Other Option

After deep analysis of all 9 problem statements against three dimensions — current AI/ML industry trends, feasibility within a hackathon timeline, and career-building value — Topic 9 (AI Money Mentor) emerges as the clear winner for your team. Here is the structured breakdown:

Topic	Trend Score	Feasibility	Career Value	Verdict
#2 Agentic Enterprise Workflows	9/10	7/10	8/10	Good but generic
#3 Cost Intelligence	8/10	5/10	7/10	Needs enterprise data
#4 Sales & Revenue AI	8/10	6/10	7/10	CRM access needed
#6 Indian Investor AI	9/10	6/10	9/10	Very competitive
#9 AI Money Mentor	10/10	9/10	10/10	BEST — Build this!

1.2 Three Core Reasons Topic 9 Wins

Reason 1: India-Scale Problem with Massive Judges Appeal

India has 14 crore+ demat accounts but over 95% of Indians have no financial plan. The problem is viscerally real, deeply personal, and affects every judge in that room. When a product demo shows someone's actual financial gap being solved in 60 seconds, it creates an emotional connection that no enterprise workflow demo can match.

Reason 2: Perfect Gen AI + Agentic AI Combination

Topic 9 demands exactly what judges are looking for in a Gen AI hackathon: large language model reasoning for personalized advice, RAG (Retrieval Augmented Generation) for tax rules and regulatory data, multi-agent coordination for complex financial calculations, and tool use for real-time data integration. This is a full Gen AI showcase, not just a chatbot wrapper.

Reason 3: Maximum Career Building Value

The skills you build here — LLM orchestration, financial data modeling, multi-agent systems, RAG pipelines, and full-stack AI deployment — are exactly what top-tier AI companies are hiring for in 2026. This project will sit at the top of your portfolio for years.

2. Product Definition — FinSaarthi

FinSaarthi: India's First Agentic Financial Life OS

An AI-powered multi-agent financial intelligence platform that acts as a personal CFO for every Indian — transforming raw financial data into personalized, actionable roadmaps across savings, investments, tax, insurance, and retirement planning. Powered by GenAI + Agentic AI, it handles the complete financial lifecycle autonomously.

2.1 Core Sub-Problems Selected (4 of 6)

Your team should implement these four sub-problems as integrated modules within one unified product:

#	Module	What It Does	Why Include It
1	MF Portfolio X-Ray	Upload CAMS/KFintech statement -> instant portfolio analysis: XIRR, overlap, expense drag, rebalancing plan	Demo-ready, visually impressive, immediate value
2	FIRE Path Planner	Input age/income/goals -> month-by-month SIP roadmap with asset allocation + insurance gap detection	Complex multi-step reasoning showcases agentic AI
3	Tax Wizard	Upload Form 16 -> AI finds missed deductions, compares old vs new regime, suggests investments by risk profile	Form parsing + RAG over tax rules = full Gen AI demo
4	Couple's Money Planner	Both partners input data -> AI optimizes across dual incomes: HRA, NPS, SIP splits, combined net worth	Unique differentiator — no other team will build this

2.2 Refined Problem Statement

> *India's 95%+ retail investors — especially dual-income couples, young professionals, and first-generation investors — have no access to personalized, affordable financial planning. Traditional financial advisors charge Rs. 25,000+ per year and are inaccessible to most. The result: missed tax deductions, unoptimized SIP strategies, insurance gaps, and zero retirement planning. FinSaarthi deploys a multi-agent AI system that understands your complete financial life in one conversation and delivers a CFO-grade financial roadmap — personalized, explainable, and actionable — at zero cost.*

3. System Architecture — Multi-Agent Design

3.1 High-Level Architecture Overview

FinSaarthi uses a 5-layer architecture combining an orchestrator agent with four specialized domain agents, a RAG knowledge base, and external tool integrations. Every decision is logged for auditability.

ARCHITECTURE LAYERS	
Layer 1	**USER INTERFACE LAYER — React.js Frontend + Streamlit Demo** Chat interface, form inputs (income/goals/portfolio upload), real-time output display with charts, downloadable reports
Layer 2	**ORCHESTRATOR AGENT — Master Planner (LangGraph/CrewAI)** Routes tasks to specialized agents, manages context window, maintains conversation history, handles errors and retries, logs all decisions to audit trail
Layer 3	**SPECIALIZED AGENT LAYER (4 Domain Agents)** Portfolio Agent: Parses CAMS PDF, calculates XIRR, detects overlap, generates rebalancing FIRE Agent: Runs compound growth simulations, builds month-by-month SIP plan, checks insurance Tax Agent: RAG over Income Tax Act, compares regimes, finds deductions, models 80C/80D/HRA Couple Agent: Dual-income optimizer, splits HRA/NPS/SIP for max tax efficiency jointly
Layer 4	**RAG KNOWLEDGE BASE — Financial Intelligence Layer** Indexed documents: Income Tax Act 2025, SEBI regulations, AMFI fund data, NSE historical data, LIC premium tables, RBI interest rate data — all chunked, embedded, and stored in ChromaDB/FAISS for real-time retrieval
Layer 5	**TOOLS & DATA INTEGRATION LAYER** Yahoo Finance API (fund NAV), PDF parsing (pypdf2/pdfplumber), SEBI MF data API, Python financial calculations (numpy-financial), SQLite for session storage, Redis for caching

3.2 Agent Communication Flow

The orchestrator uses a Directed Acyclic Graph (DAG) pattern. User input enters → Orchestrator classifies intent → Routes to 1-4 specialist agents in parallel or sequence → Each agent fetches from RAG + runs calculations → Results merge back to orchestrator → Orchestrator synthesizes final response → Audit log entry created → Response delivered to UI.

> *KEY DIFFERENTIATOR: Each agent decision is logged with timestamp, input parameters, reasoning chain, tools called, and output. This creates a full audit trail — exactly what judges score for in agentic AI evaluation.*

4. Technology Stack — Complete Guide

4.1 Core Stack Selection

Category	Technology	Version/Details	Purpose
LLM Backend	Google Gemini 1.5 Pro	Free API via AI Studio	Main reasoning engine (free tier)
LLM Alternative	Groq + Llama 3.1 70B	Free tier, ultra-fast	Fallback + fast inference
Agent Framework	LangGraph	v0.2+	Multi-agent orchestration with state graphs
Agent Tools	LangChain	v0.3+	Tool use, memory, prompt templates
RAG Vector DB	ChromaDB	Local, free	Store + retrieve financial knowledge
Embeddings	sentence-transformers	all-MiniLM-L6-v2	Free local embeddings for RAG
PDF Parsing	pdfplumber + PyPDF2	Latest	CAMS/KFintech/Form 16 parsing
Financial Math	numpy-financial	Latest	XIRR, NPV, SIP calculations
Data Processing	pandas + numpy	Latest	Portfolio analysis, data manipulation
Frontend	Streamlit (Demo)	Latest	Rapid hackathon demo UI
API Backend	FastAPI	Latest	REST endpoints for all agents
Charts/Viz	Plotly + Matplotlib	Latest	Portfolio charts, FIRE projections
Database	SQLite + Redis	Local	Session storage + caching
Deployment	Streamlit Cloud / HF Spaces	Free tier	Public demo URL for judges

4.2 Why This Stack is Perfect

- All free/open-source — zero infrastructure cost during hackathon
- Gemini 1.5 Pro: 1M token context window = perfect for large financial documents
- LangGraph: Industry-standard for production multi-agent systems — judges recognize it
- ChromaDB: Runs locally, no setup, perfect for hackathon speed
- Streamlit: Build a beautiful working demo in hours, not days

4.3 Installation Commands

Run this single command to set up your environment:

```
# Create virtual environment
python -m venv finsaarthi-env && source finsaarthi-env/bin/activate


# Install all dependencies
pip install langchain langgraph langchain-google-genai chromadb sentence-
transformers pdfplumber pypdf2 numpy-financial pandas numpy plotly
streamlit fastapi uvicorn python-dotenv


# Set API keys in .env file
GOOGLE_API_KEY=your_gemini_key_here
GROQ_API_KEY=your_groq_key_here
```

5. Step-by-Step Implementation Guide

Phase 1: Foundation Setup (Day 1 — Hours 0 to 8)

Step 1.1 — Project Structure

Create the following folder structure in your repository:

```
finsaarthi/
  agents/
    orchestrator.py       # Master agent + LangGraph state machine
    portfolio_agent.py    # MF Portfolio X-Ray logic
    fire_agent.py         # FIRE Path Planner logic
    tax_agent.py          # Tax Wizard logic
    couple_agent.py       # Couple Money Planner logic
  rag/
    knowledge_base.py     # ChromaDB setup + document ingestion
    documents/            # Tax rules, SEBI docs, fund data PDFs
  tools/
    pdf_parser.py         # CAMS + Form 16 PDF extraction
    financial_calc.py     # XIRR, SIP, NPV calculations
    market_data.py        # Yahoo Finance + SEBI API integration
  app.py                  # Streamlit main application
  api.py                  # FastAPI backend
  audit_logger.py         # Decision audit trail
  requirements.txt        # All dependencies
```

Step 1.2 — Build the Financial Calculator (Core Engine)

This is the heart of the product. Get this right first before any AI integration:

```python
# tools/financial_calc.py
import numpy_financial as npf
import numpy as np
from datetime import datetime


def calculate_xirr(cashflows: list, dates: list) -> float:
    """Calculate Extended IRR for irregular SIP cashflows"""
    # Converts cashflows + dates to XIRR using scipy root finding
    return xirr_value  # Returns annualized return rate


def calculate_sip_for_goal(goal_amount, years, rate, current_savings=0):
    """Monthly SIP needed to reach goal at given return rate"""
    months = years * 12
    monthly_rate = rate / 12
    future_current = current_savings * (1 + monthly_rate)**months
    remaining = goal_amount - future_current
    sip = npf.pmt(monthly_rate, months, 0, -remaining)
    return round(sip, 2)


def compare_tax_regimes(gross_salary, deductions_80c, hra, nps, medical):
    """Compare old vs new tax regime and return savings"""
    # Apply 2024-25 tax slabs for both regimes
    old_tax = calculate_old_regime(gross_salary, deductions_80c, hra, nps,
medical)
```

```
    new_tax = calculate_new_regime(gross_salary)
    return {'old': old_tax, 'new': new_tax, 'savings': abs(old_tax -
new_tax)}
```

Phase 2: RAG Knowledge Base (Day 1 — Hours 8 to 16)

Step 2.1 — Collect and Index Financial Documents

Download and prepare these documents for your RAG knowledge base:

- Income Tax Act 2024-25 — Download from incometax.gov.in (free)
- SEBI Mutual Fund Regulations — From sebi.gov.in (free PDF)
- AMFI Fund Performance Data — From amfiindia.com (CSV download, free)
- Insurance premium tables — LIC publishes publicly
- RBI repo rate history — RBI website, public data

Step 2.2 — Build the RAG Pipeline

```
# rag/knowledge_base.py
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings


def build_knowledge_base(docs_folder: str):
    embeddings = HuggingFaceEmbeddings(
        model_name="sentence-transformers/all-MiniLM-L6-v2"
    )  # Free, runs locally
    splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
chunk_overlap=200)
    # Load all PDFs from docs folder
    all_chunks = []
    for pdf_file in Path(docs_folder).glob('*.pdf'):
        loader = PyPDFLoader(str(pdf_file))
        chunks = splitter.split_documents(loader.load())
        all_chunks.extend(chunks)
    vectordb = Chroma.from_documents(all_chunks, embeddings,
persist_directory='./chromadb')
    return vectordb
```

Phase 3: Building the 4 Specialist Agents (Day 2 — Hours 16 to 32)

Agent 1: MF Portfolio X-Ray Agent

This is your showstopper demo feature. Build it first.

1. Parse the CAMS/KFintech PDF using pdfplumber to extract: fund names, investment dates, amounts, current NAV, redemptions
2. Calculate XIRR for each fund and overall portfolio using the financial_calc module
3. Detect portfolio overlap by comparing fund holdings (use AMFI data from your RAG)
4. Calculate expense ratio drag: annual fees eaten from returns

5. Generate a rebalancing plan using LLM: prompt Gemini with portfolio data + user risk profile + retrieved RAG context
6. Output: Plotly pie chart (allocation), bar chart (XIRR by fund), table (overlap matrix), text recommendations

Agent 2: FIRE Path Planner Agent

Multi-step reasoning agent — this is where agentic AI shines:

7. Input collection: Age, income, monthly expenses, existing investments, life goals (home, education, retirement)
8. Inflation adjustment: Calculate real return rate (nominal rate minus inflation)
9. Goal decomposition: For each goal, calculate target corpus in today's money then future value
10. SIP calculation: Use calculate_sip_for_goal for each goal separately, then show consolidated monthly SIP
11. Asset allocation: Age-based allocation (100 minus age in equity, rest in debt) — LLM refines based on risk profile
12. Insurance check: RAG retrieves thumb rule (income x 10 for life cover) — flags gap if insufficient
13. Output: Month-by-month roadmap table + Plotly line chart showing wealth growth projection to retirement

Agent 3: Tax Wizard Agent

14. PDF parsing: Extract Form 16 data — gross salary, TDS, employer PF, allowances
15. RAG retrieval: Fetch current 80C/80D/80CCD/HRA rules from your knowledge base
16. Deduction audit: For each section, check if user is maximizing — flag missed deductions
17. Regime comparison: Run compare_tax_regimes and show exact rupee savings
18. Investment suggestions: Based on tax bracket and risk profile, recommend ELSS vs PPF vs NPS vs NSC
19. Output: Two-column comparison table (Old vs New regime) + deductions checklist + savings chart

Agent 4: Couple's Money Planner Agent (Your Differentiator)

This is what no other team will build. This is why you win.

20. Dual input: Both partners enter income, investments, expenses, tax paid separately
21. HRA optimization: Check whose name the rent agreement should be — maximize HRA exemption between both
22. NPS split: Model 80CCD(1B) for both — if both contribute Rs.50,000 each, save Rs.30,000+ in taxes
23. SIP naming: Decide which goals should be in whose name for tax efficiency (LTCG harvesting)
24. Joint goal planning: Home EMI vs rent analysis, education SIP, combined retirement corpus
25. Combined net worth tracker: Monthly snapshot of total assets minus liabilities for the household
26. Output: Side-by-side partner dashboard + combined optimization score + action items for both

Phase 4: Orchestrator + LangGraph State Machine (Day 2 — Hours 32 to 40)

This is where you show multi-agent coordination — a key evaluation criterion:

```
# agents/orchestrator.py — LangGraph State Machine
from langgraph.graph import StateGraph, END
from typing import TypedDict, List


class FinancialState(TypedDict):
    user_input: str
    intent: str                      # classified intent
    user_profile: dict                # collected user data
    agent_results: dict               # results from specialist agents
    audit_log: List[dict]             # decision trail
    final_response: str               # synthesized output


# Build the graph
workflow = StateGraph(FinancialState)
workflow.add_node('classify_intent', classify_intent_node)
workflow.add_node('collect_profile', collect_profile_node)
workflow.add_node('portfolio_agent', portfolio_agent_node)
workflow.add_node('fire_agent', fire_agent_node)
workflow.add_node('tax_agent', tax_agent_node)
workflow.add_node('couple_agent', couple_agent_node)
workflow.add_node('synthesize', synthesize_response_node)
workflow.add_node('log_audit', audit_logging_node)  # CRITICAL for scoring
# Conditional routing based on intent classification
workflow.add_conditional_edges('classify_intent', route_to_agent, {...})
```

Phase 5: Streamlit UI + Demo Polish (Day 2-3 — Hours 40 to 56)

Your UI is your pitch. Make it look like a real product, not a hackathon project.

- Use st.tabs() for the 4 modules — clean navigation
- st.file_uploader() for CAMS PDF and Form 16 upload
- st.columns() for side-by-side couple planner inputs
- st.plotly_chart() for all visualizations — XIRR bars, wealth projection lines, allocation pies
- st.expander() for 'View Agent Reasoning' — shows audit log (judges love this)
- st.metric() cards for key numbers: XIRR, Tax Saved, Monthly SIP needed, Net Worth
- st.progress() bars during AI processing to simulate real-time agent activity

6. Team Member Role Assignment

6.1 Recommended 4-Member Division

Team Member	Role	Responsibilities	Hours Focus
Member 1	AI/Agent Lead	Orchestrator + LangGraph state machine, intent classification, agent communication, audit logging	Phase 3 + Phase 4
Member 2	Financial Engine	financial_calc.py XIRR/SIP/tax math, PDF parsing, FIRE calculations, couple optimization logic	Phase 1 + Agent 1,2,4
Member 3	RAG + Knowledge	ChromaDB setup, document ingestion, embedding pipeline, Tax Agent RAG queries, prompt engineering	Phase 2 + Agent 3
Member 4	UI + Integration	Streamlit app, Plotly charts, FastAPI endpoints, deployment to cloud, pitch video, README	Phase 5 + Demo

Note: Members 1 and 2 should sync every 4 hours. Member 4 should start building the UI shell on Day 1 with dummy data so it is ready for integration on Day 2. All members contribute to the 3-minute pitch video on Day 3.

7. Innovation Angles That Win Judges

7.1 What Most Teams Will Build vs What You Should Build

Average Team Will Build This	Your Team Builds This Instead
A chatbot that answers finance questions	A multi-agent system that takes action — calculates, decides, plans
Generic advice: 'save 20% of income'	Personalized math: 'You need Rs.14,200/month SIP in ELSS to retire at 50'
Single module (just Tax or just SIP)	Integrated 4-module product with shared user profile across agents
No audit trail	Full decision log: every agent action timestamped and explainable
Static demo with hardcoded data	Live demo: upload your real CAMS PDF and get real analysis in 10 seconds
Individual financial planning only	Couple optimization — unique, differentiated, judges will remember it

7.2 The Showstopper Demo Moment

> *During your 3-minute pitch: Upload a REAL CAMS statement (prepare a dummy one with realistic data). Show the Portfolio X-Ray agent run in real time. Display XIRR, overlap analysis, and rebalancing in under 15 seconds. This single moment — a real PDF going in, real personalized financial intelligence coming out — will make judges remember your team above all others.*

7.3 Gen AI Concepts Demonstrated (Judges' Checklist)

- RAG (Retrieval Augmented Generation): Tax rules retrieved from indexed financial documents
- Multi-Agent Orchestration: 4 specialized agents + 1 orchestrator coordinating via LangGraph
- Tool Use / Function Calling: LLM calls financial calculator tools, PDF parser, market data API
- Chain of Thought Reasoning: Each agent shows its reasoning steps before giving output
- Context-Aware Memory: User profile shared across all agent calls within session
- Structured Output Parsing: LLM returns structured JSON that maps to UI components
- Guardrails: Input validation, hallucination detection (if LLM suggests impossible returns, flag it)

8. Business Impact Model — The Math

8.1 Quantified Impact Estimates

Every judge wants to see numbers. Here is your impact model with assumptions:

Impact Area	Conservative Est.	Optimistic Est.	How Calculated
Tax savings per user per year	Rs. 18,000	Rs. 45,000	Avg missed 80C/80D deductions
Expense ratio reduction	Rs. 8,400/year	Rs. 21,000/year	Moving from 2.5% to 0.5% funds
SIP optimization gain over 10 years	Rs. 2.1 lakh	Rs. 8.4 lakh	Better asset allocation compounding
Time saved vs manual planning	12 hours/year	20 hours/year	CA consultation + research time
Cost saved vs financial advisor	Rs. 15,000/year	Rs. 50,000/year	Advisor fees vs zero cost AI
Couple tax optimization	Rs. 30,000/year	Rs. 75,000/year	Combined HRA + NPS dual claiming

> *Total quantifiable value per couple using FinSaarthi vs no planning: Rs. 75,000 to Rs. 2,00,000 in the first year alone. Over 10 years with compounding benefits, this exceeds Rs. 15 lakhs per household. Multiplied by 14 crore demat account holders — addressable financial value exceeds Rs. 20 lakh crore.*

8.2 Product-Market Fit Evidence

- 14 crore demat accounts in India — all potential users
- Rs. 25,000+ per year charged by human financial advisors — clear price-to-zero opportunity
- ET Markets already has 8 crore monthly users — distribution is built-in
- SEBI mandating financial literacy — regulatory tailwind for this product
- Dual income couples are the fastest growing investor segment in India (2024 SEBI report)

9. Submission Requirements — Checklist

9.1 GitHub Repository Structure

Your public GitHub repository MUST contain all of the following:

Done	Requirement	Details
[ ]	README.md	Setup instructions, architecture overview, demo link, team info, tech stack
[ ]	requirements.txt	All Python dependencies with version pinning
[ ]	Folder structure	agents/, rag/, tools/, app.py, api.py as defined in Phase 1
[ ]	Commit history	Minimum 20+ commits showing progressive build — do not push everything at the end
[ ]	Architecture diagram	1-2 page PDF: agent roles, communication flow, RAG pipeline, tool integrations
[ ]	.env.example	Shows all required API keys without actual values
[ ]	Demo data	Sample CAMS PDF and Form 16 with dummy data for reproducible demo
[ ]	Impact model	PDF or markdown file with quantified business impact (Section 8 of this report)

9.2 3-Minute Pitch Video Script

Structure your 3-minute video exactly as follows:

27. Problem Hook — 0:00 to 0:30: Show the stat '95% of Indians have no financial plan.' Show a couple struggling with financial decisions. Make it emotional.
28. Solution Intro — 0:30 to 0:50: Introduce FinSaarthi. One sentence: 'AI that acts as your personal CFO.'
29. Live Demo Module 1 — 0:50 to 1:20: Upload CAMS PDF. Show Portfolio X-Ray result in real time. Highlight XIRR and rebalancing suggestions.
30. Live Demo Module 2 — 1:20 to 1:45: Show FIRE Planner with sample inputs. Display the wealth projection chart and monthly SIP breakdown.
31. Live Demo Module 3 — 1:45 to 2:05: Tax Wizard — show Rs. 45,000 in missed deductions found in 8 seconds.
32. Live Demo Module 4 — 2:05 to 2:25: Couple Planner — show dual income optimization saving Rs. 60,000 in taxes jointly.
33. Architecture + Agent Reasoning — 2:25 to 2:45: Quick screen showing LangGraph flow, audit log, and RAG retrieval in action.
34. Impact + Close — 2:45 to 3:00: Show the impact numbers. End with: 'FinSaarthi — financial freedom for every Indian.'

9.3 Architecture Document Template

Your 1-2 page architecture document must include:

- System overview diagram: Show all 5 layers (UI, Orchestrator, Agents, RAG, Tools)
- Agent communication diagram: Arrows showing how orchestrator routes to each agent
- Data flow diagram: User input → processing steps → output delivery
- RAG pipeline diagram: Document ingestion → chunking → embedding → retrieval → LLM → response
- Error handling: Show what happens when a PDF is malformed or API fails
- Tools table: List every external API, library, and data source used

10. Career Building Value — Why This Project Matters

10.1 Skills You Build That Are Directly Hireable

Skill Built	Industry Demand (2026)	Where It Gets You Hired
LangGraph Multi-Agent Systems	Extremely High — new standard	AI startups, Anthropic, OpenAI partners
RAG Pipeline Engineering	Very High — every enterprise AI needs this	TCS, Infosys AI divisions, fintechs
LLM Prompt Engineering	High — foundational GenAI skill	Any company building AI products
Financial Data Modeling	High — FinTech is booming in India	Zerodha, Groww, ET Money, CRED
FastAPI + Streamlit	Medium-High — fast prototyping skills	AI product companies, startups
PDF Parsing + Data Extraction	High — document AI is growing fast	Insurance, banking, legal tech
Full End-to-End AI Product	Very High — rare complete skill	Senior roles, founding engineer roles

10.2 How to Present This on Your Resume

> *FinSaarthi — AI Financial Planning System | ET AI Hackathon 2026: Built a multi-agent financial intelligence platform using LangGraph, Gemini 1.5 Pro, and RAG (ChromaDB). Deployed 4 specialized AI agents (Portfolio Analysis, FIRE Planning, Tax Optimization, Couple Finance) with full audit trail. Analyzed MF portfolios via CAMS PDF parsing, calculating XIRR and generating personalized rebalancing recommendations. Quantified Rs. 75,000+ annual financial impact per user through tax optimization and SIP strategy improvements.*

10.3 LinkedIn Project Post Template

After the hackathon, post this on LinkedIn:

> *Just shipped FinSaarthi at ET AI Hackathon 2026 — an agentic AI system that acts as India's personal financial CFO. Built a 5-agent LangGraph system with RAG over financial regulations, real CAMS PDF analysis with XIRR calculation, tax regime optimization, and India's first AI-powered couple financial planner. Every decision is logged in an audit trail. Full source code on GitHub. Looking forward to feedback from the fintech community!*

11. 72-Hour Hackathon Game Plan

Hour-by-Hour Timeline

Hours	Phase	What Gets Built	Owner
0-4	Setup	Repo, venv, API keys, folder structure, dummy CAMS PDF prepared	All members
4-10	Core Engine	financial_calc.py: XIRR, SIP, tax regime comparison — fully tested	Member 2
4-10	RAG Setup	ChromaDB running, 5 financial documents indexed and queryable	Member 3
4-10	UI Shell	Streamlit app with 4 tabs, file uploaders, dummy data displayed	Member 4
10-16	Agent 1	Portfolio X-Ray: PDF parse + XIRR + Plotly charts — working end to end	Members 1+2
16-22	Agent 2	FIRE Planner: Goal inputs + SIP calc + projection chart working	Members 1+2
16-22	Agent 3	Tax Wizard: Form 16 parsing + regime comparison + RAG deduction lookup	Members 1+3
22-28	Agent 4	Couple Planner: Dual input + HRA/NPS optimization + joint SIP split	Members 2+3
28-36	Orchestrator	LangGraph state machine routing all 4 agents, audit logging running	Member 1
36-44	Integration	All agents connected to UI, end-to-end flow tested, bugs fixed	All members
44-52	Polish	UI styling, error handling, loading states, mobile responsive check	Member 4
52-58	Deployment	Deploy to Streamlit Cloud or HF Spaces, get public URL, test on mobile	Member 4
58-64	Documentation	README, architecture diagram, impact model PDF completed	Member 3
64-70	Pitch Video	Record 3-minute demo video following the script in Section 9.2	All members
70-72	Submit	Final check of GitHub repo, video upload, submission form	All members

> *CRITICAL WARNING: Never push all your code in one commit at hour 71. Judges and evaluators look at commit history as a sign of genuine work. Start committing from Hour 1 — every feature addition, every bug fix should be a separate commit with a descriptive message.*

12. Common Pitfalls — Avoid These

Mistake	How to Avoid It
Building a chatbot instead of an agent system	Use LangGraph explicitly. Show agent nodes on screen during demo.
Hallucinated financial numbers	Hard-code the math in Python, use LLM only for narrative explanation, not calculation.
Demo crashes during pitch	Test demo 10+ times. Have screenshot backup slides. Pre-record a fallback video.
Scope creep — trying to build all 6 sub-problems	Build 4 modules excellently. Judges prefer depth over breadth.
No quantified impact numbers	Section 8 of this report is your answer. Memorize those numbers.
Using GPT-4 with a paid key that expires	Use Gemini via AI Studio — generous free tier, no credit card, does not expire.
Ignoring the audit trail requirement	Log EVERY agent decision from Day 1. Build the logger before any agent logic.
Weak problem statement in pitch	Use the exact statement from Section 2.2 of this report — it is research-backed.

13. How Judges Will Score You

13.1 Mapped to ET Hackathon Evaluation Criteria

Judging Criterion	Your Score (Est.)	How You Score High
Full Workflow Automation	9/10	All 4 modules run end-to-end autonomously: PDF in, recommendations out
Multi-Agent Coordination	10/10	LangGraph orchestrator + 4 specialist agents is textbook multi-agent architecture
Measurable Reduction in Cycle Time	9/10	Manual: 12 hours research + CA visit. FinSaarthi: 60 seconds. That is 720x faster.
Auditability of Decisions	10/10	Every agent decision logged with timestamp, input, reasoning, and output
Quantified Financial Impact	9/10	Rs. 75,000+ per user per year, backed by calculation logic in the app
Real-World Applicability	10/10	Upload your actual CAMS PDF. Real output. Real value. Not a simulation.
Innovation and Differentiation	9/10	Couple's Money Planner is completely unique — no competing product exists
Working Demo	9/10	Live demo from uploaded file to output in under 30 seconds

14. Final Message to Your Team

You are not building a hackathon project.

You are building a portfolio-defining product that demonstrates everything the AI industry is hiring for in 2026.

Multi-agent systems. RAG. LLM reasoning. Financial domain expertise. Full-stack deployment.

Build it right. Ship it working. Explain it clearly.

That is how you win the hackathon. That is how you build your career.

FinSaarthi — Har Indian Ka Apna CFO

ET AI Hackathon 2026 | Topic 9: AI Money Mentor | All the best da!